

Tema 2 - CSS y construcción de interfaces web

2.1 - Fundamentos de CSS, selectores y cascada

Saber CSS no es solo conocer propiedades

Aplicar una propiedad CSS aislada resulta sencillo:

```
button { background-color: #2266cc; }
```

La dificultad aparece cuando una interfaz crece. Un mismo botón puede recibir estilos desde una regla general, una clase de componente, otra clase utilizada para indicar su variante y una regla específica de una zona concreta. Entonces aparecen preguntas más importantes:

- ¿Por qué se aplica una regla y no otra?
- ¿Qué selector conviene utilizar?
- ¿Qué ocurre si dos reglas modifican la misma propiedad?
- ¿Cómo podemos cambiar el aspecto de muchos elementos sin repetir código?
- ¿Cómo evitamos efectos inesperados en otras páginas?

CSS no funciona como una lista de instrucciones que se ejecutan de arriba abajo. El navegador debe determinar qué reglas afectan a cada elemento y resolver posibles conflictos. Para trabajar con CSS de forma consistente necesitamos dominar tres mecanismos:

Selección

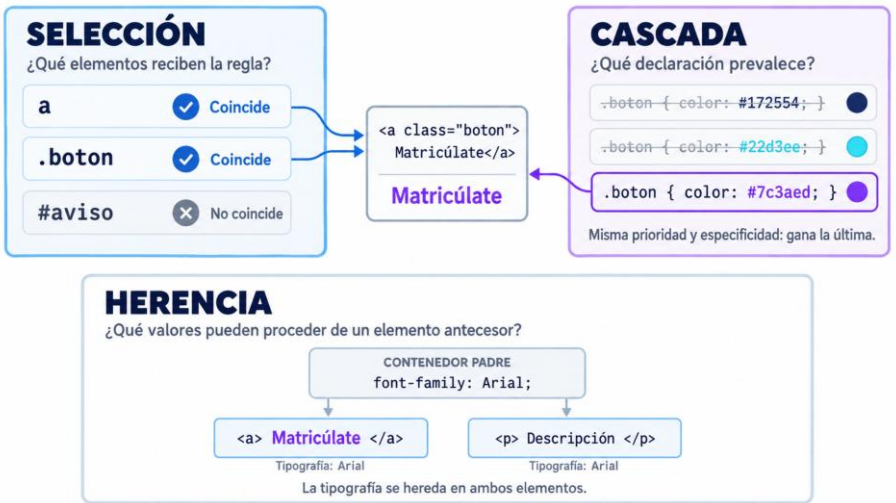
Determina qué elementos reciben una regla.

Cascada

Resuelve qué declaración prevalece cuando varias compiten por una misma propiedad.

Herencia

Permite que determinados valores se transmitan desde un elemento a sus descendientes.



En este apartado nos centraremos en estos mecanismos. Las propiedades de presentación, el modelo de cajas y la maquetación se desarrollarán después.

> Conocer muchas propiedades CSS sirve de poco si no podemos predecir con precisión dónde se aplicarán y cuál prevalecerá.

Tres formas de incorporar CSS

CSS puede asociarse a un documento HTML de diferentes maneras. Las tres formas pueden producir un resultado visual similar, pero no tienen la misma utilidad dentro de un proyecto.

Estilo en línea

La declaración se incorpora directamente al atributo `style` del elemento.

```
<a href="cursos.html"
   style="color: #2266cc; font-weight: 700;">
  Cursos
</a>
```

Puede resultar útil en situaciones muy concretas, pero mezcla estructura y presentación y dificulta la reutilización.

Hoja interna

Las reglas se escriben dentro de un elemento `style`, normalmente situado en `head`.

```
<style>
  .enlace-principal {
    color: #2266cc;
    font-weight: 700;
  }
</style>
```

Puede ser útil en un documento aislado o en una demostración pequeña, pero sus reglas permanecen vinculadas a esa página.

Hoja externa

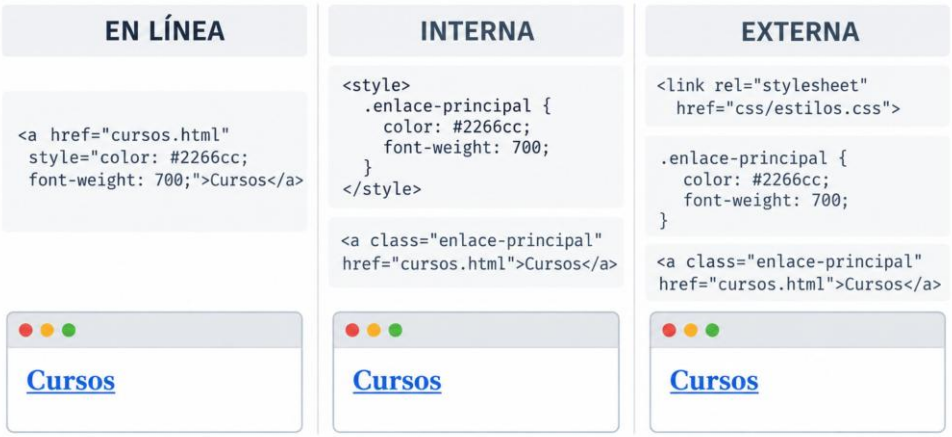
Las reglas se almacenan en un archivo independiente.

```
<link rel="stylesheet"
      href="css/estilos.css">
```

```
.enlace-principal {
  color: #2266cc;
  font-weight: 700;
}
```

Es la opción habitual cuando varias páginas comparten un sistema visual. Permite reutilizar reglas y mantener la estructura HTML separada de la presentación.

No se trata de que una forma sea siempre válida y las demás estén prohibidas. La elección depende del contexto. En un sitio compuesto por varias páginas, las hojas externas permiten mantener la presentación de forma mucho más coherente.



Lo que cambia es dónde están las reglas, no el resultado.

Vincular correctamente una hoja externa

Una hoja externa solo se aplicará si el navegador puede localizarla. Supongamos esta estructura:

```
aula-abierta/
│
├── index.html
├── cursos.html
│
└── css/
    └── estilos.css
```

Tanto `index.html` como `cursos.html` pueden utilizar la misma hoja:

```
<link rel="stylesheet"
      href="css/estilos.css">
```

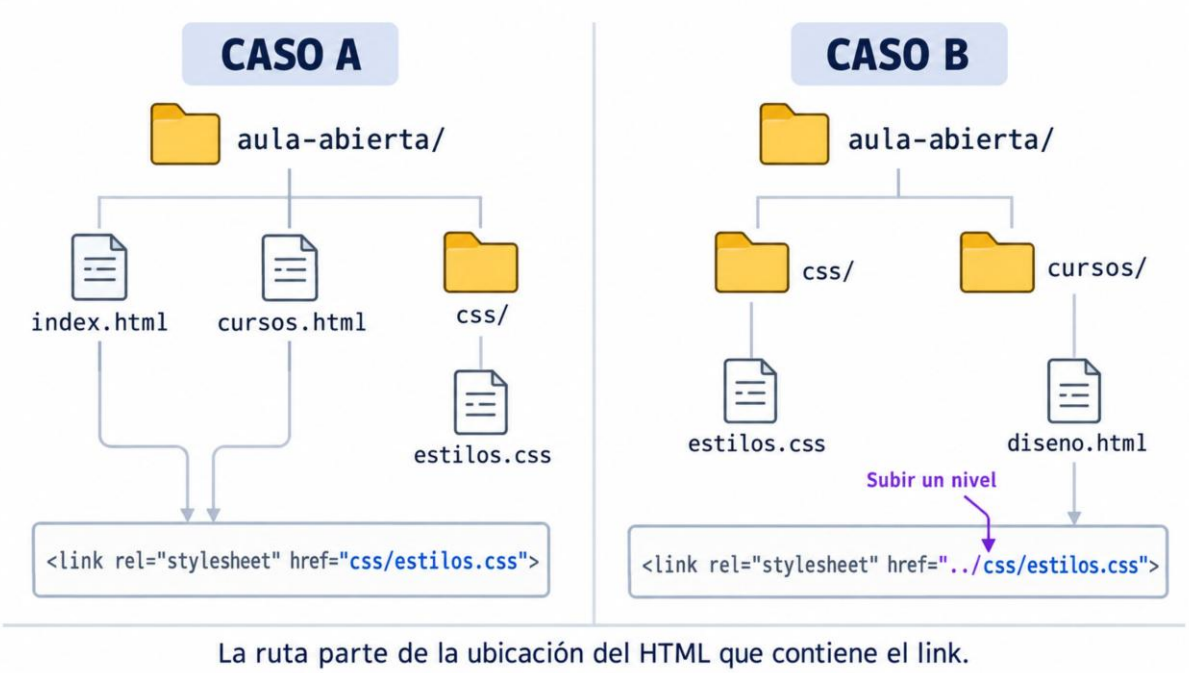
La ruta se interpreta desde el documento HTML que contiene el elemento `link`.

Si una página estuviera dentro de otra carpeta:

```
aula-abierta/
│
├── css/
│   └── estilos.css
│
└── cursos/
    └── diseno.html
```

la ruta relativa tendría que adaptarse:

```
<link rel="stylesheet"
      href="../css/estilos.css">
```



Un error frecuente consiste en modificar CSS cuando el verdadero problema es que el archivo no se está cargando. Antes de revisar selectores o propiedades conviene comprobar:

- que la ruta sea correcta;
- que el nombre del archivo coincida;
- que la extensión sea correcta;
- que el navegador haya cargado realmente el recurso.

Las herramientas de desarrollo del navegador permiten comprobarlo sin necesidad de modificar el código a ciegas.

Selectores: elegir qué vamos a modificar

Un selector determina qué elementos del documento recibirán las declaraciones de una regla. Partimos de este HTML:

```
<article class="tarjeta-curso">
  <h2>Diseño de Interfaces Web</h2>
  <p class="descripcion">
    Aprende a planificar y construir interfaces web.
  </p>
  <a class="boton" href="curso-diseno.html">
    Ver curso
  </a>
</article>
```

Por tipo de elemento

```
h2 { color: #172554; }
```

Por clase

```
.descripcion { color: #475569; }
```

Por identificador

```
#contenido-principal {
  max-width: 75rem;
}
```

Un identificador `id` debe ser **único** dentro del documento. El selector `#id` permite seleccionar ese elemento, aunque para construir componentes reutilizables suelen resultar más flexibles las clases.

En una interfaz reutilizable, las **clases** suelen proporcionar la mayor flexibilidad para aplicar estilos a componentes y variantes.



Selector universal

El selector universal `*` representa **cualquier elemento HTML** en el documento. Permite seleccionar **cualquier elemento** que se encuentre dentro del ámbito indicado por el selector, lo que lo convierte en una herramienta potente para establecer un comportamiento global.

```
* {
  box-sizing: border-box;
}
```

En este ejemplo, se aplica la propiedad `box-sizing: border-box;` a todos los elementos del HTML. Esta es una práctica común en el desarrollo web moderno para controlar cómo se calculan el ancho y el alto de los elementos, simplificando el diseño de las cajas.

Además, el selector universal puede combinarse con otros selectores para restringir su alcance a un contexto específico:

El selector universal también puede formar parte de un selector más concreto.

```
.tarjeta-curso * {
  font-family: inherit;
}
```

Aquí, el selector `.tarjeta-curso *` no selecciona todos los elementos de la página, sino únicamente aquellos que son descendientes de un elemento con la clase `.tarjeta-curso`. Esto permite aplicar estilos de manera más granular, por ejemplo, asegurando que todos los elementos dentro de una tarjeta específica hereden la fuente de su padre sin afectar al resto del documento.



USOS HABITUALES

Se utiliza con frecuencia para establecer reglas generales predeterminadas o como parte de un proceso de normalización inicial de estilos, lo que ayuda a crear una base consistente antes de aplicar estilos más específicos.

```
* {
    margin: 0;
    padding: 0;
    box-sizing: border-box;
}
```

Idea fundamental: El asterisco no significa necesariamente «toda la página». Significa «cualquier elemento que cumpla el resto de condiciones del selector».

Una clase puede reutilizarse

Una clase CSS describe una característica que puede aparecer en diferentes elementos.

Clase base compartida

```
.boton {
  display: inline-block;
  padding: 0.75rem 1rem;
  border-radius: 0.5rem;
  text-decoration: none;
}
```

La misma clase puede utilizarse varias veces:

```
<a class="boton" href="cursos.html"> Ver
cursos
</a>
<a class="boton" href="contacto.html">
  Contactar
</a>
```

Composición con variante

```
<a class="boton boton--principal"
   href="matricula.html">
  Matricúlate
</a>
```



La primera clase define características compartidas. La segunda introduce una variante. Esta composición evita crear una regla completamente nueva para cada botón.

Una clase no tiene que corresponder necesariamente con un único elemento HTML. Puede utilizarse en cualquier elemento cuya función o presentación justifique esa regla. El nombre de la clase debe describir una función o concepto suficientemente estable. Nombres como `.boton-principal` o `.tarjeta-curso` suelen resultar más mantenibles que nombres vinculados únicamente a una apariencia accidental como `.texto-azul-grande`.

Combinar selectores no es lo mismo que agruparlos

CSS permite exigir varias condiciones al mismo elemento. Por ejemplo `p.aviso` selecciona únicamente los elementos `p` que además tienen la clase `aviso`.

También podemos exigir que un elemento tenga dos clases:

```
.tarjeta-curso.destacado {
  border-color: #7c3aed;
}
```

Selecciona un mismo elemento que posee simultáneamente las clases:

```
<article class="tarjeta-curso destacado">
```

No debe confundirse con:

```
.tarjeta-curso .destacado
```

que selecciona un elemento `.destacado` situado **dentro de** `.tarjeta-curso`.

Ni debe confundirse tampoco con `.tarjeta-curso, .destacado` que agrupa dos selectores independientes.

Agrupar selectores sin repetir declaraciones

Si varios selectores necesitan exactamente una misma declaración, podemos agruparlos mediante comas.



Agrupar selectores no significa que los elementos estén relacionados dentro del HTML. Únicamente indica que comparten ese conjunto de declaraciones.

```
h1, h2, h3 {
  color: #172554;
  line-height: 1.2;
}
```

La coma indica que cada selector se evalúa de forma independiente. Los tres tipos de encabezado reciben las mismas declaraciones.

También podemos agrupar selectores más complejos:

```
.tarjeta-curso h2,
.tarjeta-noticia h2 {
  margin-top: 0;
}
```

La agrupación resulta adecuada cuando las reglas son realmente comunes. No conviene agrupar elementos únicamente porque hoy tengan el mismo aspecto si representan conceptos que probablemente evolucionarán de manera diferente. Por ejemplo:

```
.boton, .aviso-error {
  background-color: #2266cc;
}
```

puede funcionar visualmente, pero establece una dependencia poco lógica entre dos componentes diferentes.

> Evitar repetición es útil, pero una regla compartida debe expresar una relación real entre los elementos que agrupa.

Seleccionar según la estructura del documento

Los combinadores permiten seleccionar elementos según su relación dentro del HTML.

Elegir el combinador adecuado evita crear clases innecesarias y permite expresar relaciones estructurales reales.

Partimos de:

```
<article class="tarjeta-curso">
  <h2>Diseño de Interfaces Web</h2>
  <p>Curso online</p>
  <p>120 horas</p>

  <div class="datos">
    <p>120 €</p>
  </div>
</article>
```

1 Descendiente

```
.tarjeta-curso p { color: #475569; }
```

Selecciona cualquier `p` situado dentro de `.tarjeta-curso`, aunque exista más de un nivel intermedio.

2 Hijo directo

```
.tarjeta-curso > h2 { margin-top: 0; }
```

Selecciona únicamente los `h2` que son hijos directos de `.tarjeta-curso`.

3 Hermano adyacente

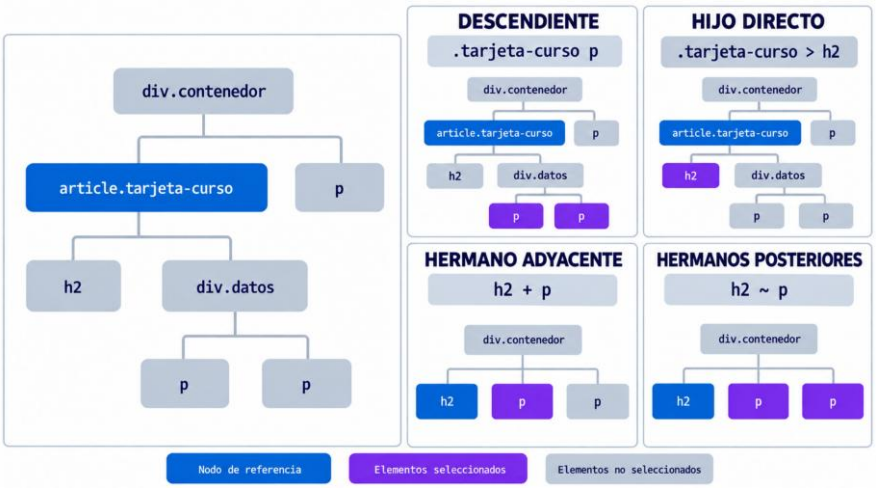
```
h2 + p { margin-top: 0.5rem; }
```

Selecciona el primer `p` situado inmediatamente después de un `h2`.

4 Hermanos posteriores

```
h2 ~ p { color: #475569; }
```

Selecciona los `p` posteriores al `h2` que comparten el mismo elemento padre.



Selectores de atributo

CSS también puede seleccionar elementos según la presencia o el valor de un atributo HTML.

Presencia del atributo

```
[required] {
  ...
}
```

Selecciona todos los elementos que tienen el atributo `required`.

Valor exacto

```
input[type="email"]
{
  ...
}
```

Selecciona los `input` cuyo atributo `type` vale exactamente `email`.

El valor comienza por

```
a[href^="https://"]
{
  ...
}
```

El valor termina en

```
a[href$=".pdf"] {
  ...
}
```

El valor contiene una cadena

```
a[href*="contacto"] {
  ...
}
```

Resumen de operadores

Selector	Selecciona elementos cuyo atributo...
`[attr]`	existe, independientemente de su valor
`[attr="x"]`	tiene exactamente el valor `x`
`[attr~="x"]`	contiene `x` como una palabra dentro de una lista separada por espacios
`[attr|="x"]`	vale `x` o comienza por `x-`
`[attr^="x"]`	comienza por `x`
`[attr$="x"]`	termina por `x`
`[attr*="x"]`	contiene la cadena `x`

Los selectores de atributo resultan útiles cuando una característica expresada en el propio HTML permite identificar el elemento de forma fiable. No deben sustituir indiscriminadamente a las clases.

Pseudoclases: seleccionar un estado o una condición

Las pseudoclases permiten seleccionar elementos cuando cumplen una condición determinada. Se escriben con un único `:`. Una pseudoclase no añade un estado nuevo al HTML. Selecciona una situación que el navegador puede reconocer a partir del documento o de la interacción.

Interacción

```
.boton:hover {
  background-color: #1d4ed8;
}

.campo:focus {
  border-color: #2266cc;
}

.boton:focus-visible {
  outline: 0.2rem solid #22d3ee;
}
```

`:focus` selecciona un elemento cuando recibe el foco. `:focus-visible` permite aplicar un indicador cuando el navegador considera necesario mostrar visualmente ese foco, algo especialmente útil durante la navegación mediante teclado.

Estado de controles

```
input:required {
  border-left: 0.25rem solid #2266cc;
}

input:disabled {
  opacity: 0.6;
}
```

Posición dentro de una estructura

```
.tarjeta-curso:first-child {
  margin-top: 0;
}

.tarjeta-curso:last-child {
  margin-bottom: 0;
}

.tarjeta-curso:nth-child(2) {
  border-color: #8b5cf6;
}

.tarjeta-curso:nth-child(3n) {
  border-color: #22d3ee;
}
```

`:nth-child()` puede seleccionar una posición concreta o seguir un patrón. Expresiones como `2n`, `2n+1` o `3n` permiten seleccionar grupos periódicos de elementos. La posición se calcula respecto al conjunto de elementos hermanos del mismo padre, no únicamente respecto a los elementos que coinciden con la clase utilizada en el selector.



Pseudoelementos `::before` y `::after`: generar contenido desde CSS

Los pseudoelementos `::before` y `::after` permiten generar una caja asociada a un elemento sin necesidad de añadir una nueva etiqueta al código HTML, `::before` genera esa caja antes del contenido propio del elemento, mientras que `::after` la genera después. La sintaxis moderna utiliza dos puntos dobles (`::`) para distinguir los pseudoelementos de las pseudoclases. En código antiguo todavía puede encontrarse la sintaxis `:before` y `:after`, pero utilizaremos `::before` y `::after`.

Para que `::before` o `::after` generen contenido visible se utiliza habitualmente la propiedad `content`. Esta propiedad puede contener texto, un símbolo o incluso una cadena vacía cuando el pseudoelemento se utiliza únicamente con una finalidad gráfica.

`::before`

```
<p class="aviso">
  Matrícula abierta.
</p>

.aviso::before {
  content: "¡ ";
  color: #2266cc;
  font-weight: 700;
}

Resultado: ¡ Matrícula abierta.
```

El símbolo no aparece en el HTML. Lo genera CSS mediante `::before` y se representa antes del texto original del párrafo.

`::after`

```
<a class="enlace-externo"
   href="https://example.com">
  Documentación
</a>

.enlace-externo::after {
  content: " #";
}

Resultado:  Documentación #
```

El contenido generado aparece después del texto del enlace sin necesidad de escribir el símbolo dentro del HTML.

El contenido esencial debe permanecer en el HTML. Los pseudoelementos son adecuados para complementar la presentación, pero no deberían ser el único lugar donde aparezca información necesaria para comprender o utilizar la interfaz.

```
.error::before {
  content: "El correo electrónico no es
válido";
}
```

Utilizar este pseudoelemento como único mensaje de error no sería una buena solución: esa información forma parte del contenido y debería existir en el documento HTML.

Pseudoelementos: seleccionar partes de un elemento

A diferencia de las pseudoclases, que seleccionan un elemento basándose en su estado o condición, los pseudoelementos permiten aplicar estilos a partes específicas de un elemento que no son accesibles mediante los selectores normales de CSS. No todos los pseudoelementos generan contenido nuevo. Algunos permiten aplicar estilos a una parte concreta de un elemento ya existente, como la primera letra o la primera línea de un párrafo.

Primera letra (`::first-letter`)

```
.introduccion::first-letter {
  ...
}
```

`::first-letter` permite seleccionar y estilizar la primera letra de la primera línea de un bloque de texto. Es ideal para crear efectos tipográficos como las mayúsculas capitulares, añadiendo un toque distintivo al inicio de un párrafo o sección.

Primera línea (`::first-line`)

```
.introduccion::first-line {
  ...
}
```

`::first-line` aplica estilos exclusivamente a la primera línea visual del texto de un elemento. Es importante recordar que la cantidad de texto incluida en esa línea puede variar dinámicamente según el ancho disponible del contenedor y el tamaño de la fuente, lo que la hace útil para destacar introducciones cortas de forma fluida.

Marcador de una lista (`::marker`)

```
li::marker {
  ...
}
```

`::marker` permite aplicar estilos al símbolo o número que actúa como marcador en los elementos de una lista (`<li>`). Esto ofrece un control granular sobre el diseño de las listas, permitiendo personalizar el color, tamaño y otros aspectos del marcador sin afectar el texto del elemento de la lista en sí.

Texto de ayuda de un campo (`::placeholder`)

```
input::placeholder {
  ...
}
```

`::placeholder` es un pseudoelemento que permite estilizar el texto provisional (hint) mostrado por algunos controles de formulario, como `<input>` o `<textarea>`. Es una herramienta esencial para mejorar la experiencia de usuario, haciendo que los placeholders sean más legibles o armónicos con el diseño general.

Diferencia fundamental entre Pseudoclases y Pseudoelementos

Aunque ambos comienzan con `:` o `::` y extienden las capacidades de los selectores, es crucial entender su distinción:

`:hover` (Pseudoclase)

Una pseudoclase selecciona un elemento completo cuando este cumple una condición específica o se encuentra en un determinado estado. Por ejemplo, `:hover` selecciona un elemento cuando el cursor del ratón está sobre él. Las pseudoclases representan estados dinámicos o estructurales del elemento.

`::first-line` (Pseudoelemento)

Un pseudoelemento selecciona una parte conceptual de un elemento, una porción que no existe como un elemento HTML independiente en el DOM, pero que el navegador renderiza como tal. Por ejemplo, `::first-line` selecciona la primera línea de texto de un párrafo, no el párrafo completo.

> Mientras que una **pseudoclase** selecciona un elemento basándose en su *estado o condición*, un **pseudoelemento** selecciona o representa una *parte específica* de un elemento.



Selectores funcionales útiles

CSS incorpora pseudoclases funcionales que permiten expresar selectores complejos de forma más clara.

:not()

Selecciona elementos que no cumplen una condición.

```
nav a:not(.activo) {
  color: #475569;
}
```

:is()

Permite agrupar diferentes posibilidades dentro de un selector.

```
:is(h1, h2, h3) a {
  color: inherit;
}
```

equivale conceptualmente a seleccionar los enlaces contenidos en cualquiera de esos encabezados.

:where()

Funciona de manera parecida a `:is()` y permite agrupar diferentes alternativas dentro de un selector.

```
:where(header, main, footer) a {
  text-decoration-thickness: 0.1em;
}
```

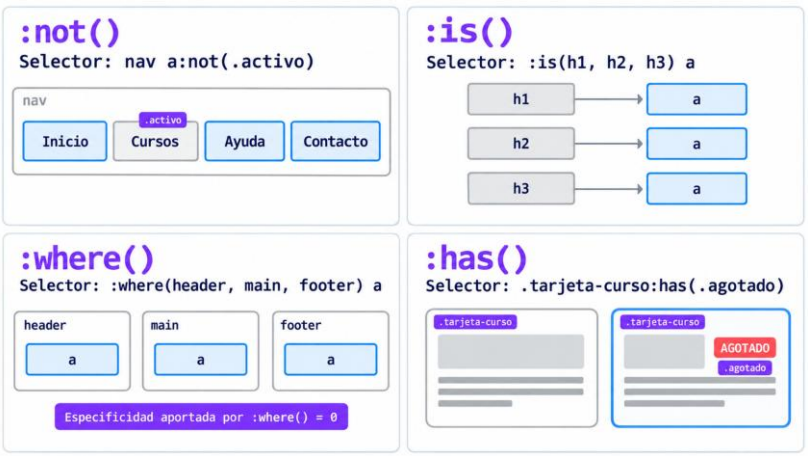
:has()

Permite seleccionar un elemento en función de lo que contiene.

```
.tarjeta-curso:has(.agotado) {
  opacity: 0.7;
}
```

Selecciona una tarjeta que contiene un elemento con clase `.agotado`.

Estos selectores permiten expresar relaciones potentes. No es necesario utilizarlos siempre: un selector sencillo continúa siendo preferible cuando resuelve correctamente el problema.



Modelo en cascada

Supongamos este elemento:

```
<a class="boton destacado"
href="matricula.html">
  Matricúlate
</a>
```

y estas reglas:

```
a          { color: #172554; }
.boton     { color: #2266cc; }
.destacado { color: #ffffff; }
```

Las tres reglas afectan al mismo elemento y modifican la misma propiedad. El navegador necesita decidir cuál prevalece. CSS utiliza la **cascada** para resolver esos conflictos.

Las declaraciones CSS pueden proceder de diferentes orígenes, como los estilos predeterminados del navegador, los estilos definidos por el autor de la página o determinadas preferencias del usuario. Además, una declaración puede ser normal o estar marcada como importante. En nuestros ejemplos trabajaremos principalmente con reglas normales definidas por el autor.

En los casos habituales que trabajaremos, debemos considerar principalmente:

01

Aplicabilidad

Si la regla puede aplicarse al elemento.

02

Origen e importancia

El origen y la importancia de las declaraciones.

03

Especificidad

La especificidad del selector.

04

Orden

El orden en el que aparecen las reglas cuando lo anterior empata.

No debemos reducir el modelo en cascada a «la última regla siempre gana». La última regla solo decide cuando las declaraciones que compiten tienen una prioridad equivalente. En el ejemplo anterior, `.boton` y `.destacado` tienen la misma especificidad. Como `.destacado` aparece después, su color prevalece.



Especificidad: no todos los selectores pesan igual

Cuando varias reglas de un mismo contexto compiten por una propiedad, la especificidad ayuda a determinar cuál prevalece. Podemos comparar los selectores según tres grupos:

A

Identificadores

B

Clases, atributos y pseudoclases

C

Elementos y pseudoelementos

Selector	Especificidad (A-B-C)	Observación
`p`	`0 - 0 - 1`	Un elemento
`.descripcion`	`0 - 1 - 0`	Una clase
`.tarjeta .descripcion`	`0 - 2 - 0`	Dos clases
`#principal .descripcion`	`1 - 1 - 0`	Un identificador y una clase

La comparación se realiza empezando por el grupo de identificadores. No se suman los valores para obtener una puntuación decimal. Por eso `1 - 0 - 0` tiene más prioridad que `0 - 20 - 0`.

Algunos selectores que ya hemos utilizado tienen un comportamiento especial, el selector universal `*` y la pseudoclase `:where()` no añaden especificidad. En cambio, en: `:is`, `:not`, `:has` la especificidad viene determinada por el selector más específico incluido en sus argumentos.

Los estilos escritos directamente en el atributo `style` tienen una prioridad propia dentro de los estilos de autor y, salvo excepciones como una declaración **!important** aplicable, prevalecen frente a las reglas normales de las hojas de estilo.

Una especificidad elevada no significa que un selector sea mejor. En un sistema reutilizable suele interesar mantenerla controlada para que los componentes puedan modificarse sin crear una cadena de selectores cada vez más específicos.

Caso práctico de especificidad

HTML

```
<section id="cursos">
  <article class="tarjeta">
    <h2 class="titulo">
      Diseño de Interfaces Web
    </h2>
  </article>
</section>
```

CSS

```
h2               { color: black;  }
.titulo          { color: blue;   }
.tarjeta .titulo { color: purple; }
#cursos .titulo  { color: green;  }
```

Comparativa de especificidades

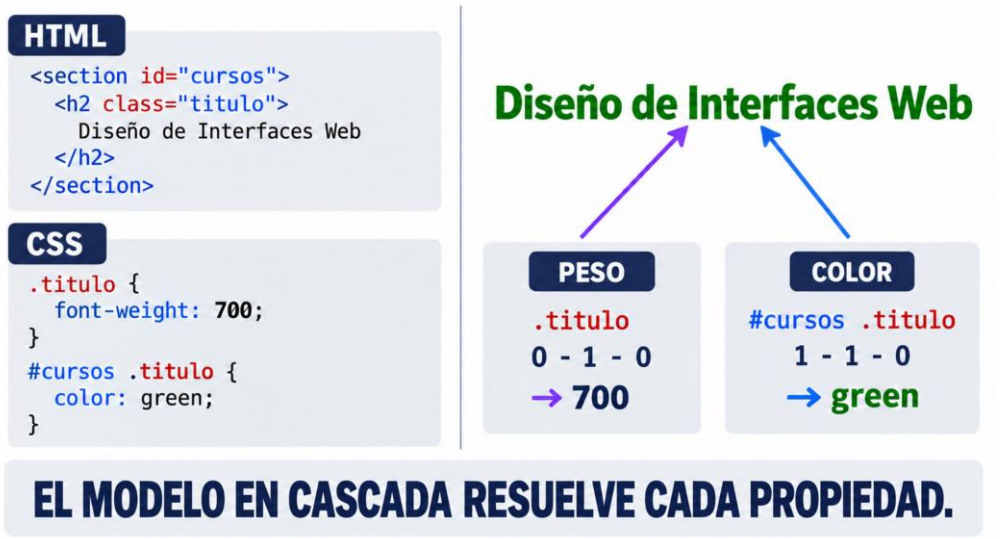
Selector	Especificidad
`h2`	`0 - 0 - 1`
`.titulo`	`0 - 1 - 0`
`.tarjeta .titulo`	`0 - 2 - 0`
`#cursos .titulo`	`1 - 1 - 0`

La declaración que finalmente establece el color es `#cursos .titulo { color: green; }` porque su selector contiene un identificador.

Ahora añadimos: `.titulo { font-weight: 700; }`

Esta regla también se aplica. La cascada no elige una regla completa y descarta todas las demás. Resuelve **cada propiedad que entra en conflicto**. El título puede recibir `color` desde una regla y `font-weight` desde otra.

Esta idea resulta fundamental para interpretar correctamente el panel de estilos calculados del navegador.



El orden solo decide cuando hay empate

Caso 1: misma especificidad

```
.boton { background-color: #2266cc; }
.boton { background-color: #7c3aed; }
```

Ambos selectores tienen la misma especificidad y pertenecen al mismo contexto. La segunda declaración prevalece:

`background-color: #7c3aed`

Caso 2: especificidad diferente

```
#accion-principal {
  background-color: #2266cc;
}
.boton {
  background-color: #7c3aed;
}
```

Aunque `.boton` aparece después, el selector `#accion-principal` tiene mayor especificidad. Por tanto, si el elemento coincide con ambos:

`background-color: #2266cc`



✗ Incorrecto

«CSS siempre aplica lo último que aparece»

✓ Correcto

Cuando dos declaraciones tienen la misma prioridad dentro del modelo en cascada, la que **aparece después** puede resolver el empate.

Comprender esta diferencia evita muchos cambios innecesarios de orden y reduce la tentación de añadir `!important` para solucionar conflictos que realmente proceden de la especificidad.

Herencia: algunas propiedades se transmiten

El modelo en cascada decide qué valor recibe una propiedad en un elemento. La herencia explica por qué determinados valores pueden transmitirse a sus descendientes.

HTML

```
<article class="tarjeta">
  <h2>Diseño de Interfaces Web</h2>
  <p>Curso de 120 horas.</p>
</article>
```

Propiedad heredada

```
.tarjeta { color: #172554; }
```

Tanto el `h2` como el `p` pueden utilizar ese color aunque no exista una regla específica para ellos.

Propiedad no heredada

```
.tarjeta {
  border: 1px solid #cbd5e1;
}
```

El borde pertenece a la tarjeta. Sus descendientes no reciben automáticamente ese borde.

Valores globales relacionados

```
color: inherit;
```

`inherit` Utiliza el valor calculado del elemento padre.

```
color: initial;
```

`initial` Utiliza el valor inicial definido para esa propiedad.

```
color: unset;
```

`unset` Se comporta como `inherit` si la propiedad se hereda normalmente y como `initial` si no se hereda.

```
color: revert;
```

`revert` Permite retroceder al valor que correspondería según un origen o nivel anterior de la cascada.

No todas las propiedades se comportan igual

Propiedades que suelen heredarse	Propiedades que normalmente no se heredan
color	margin
font-family	padding
font-size	border
line-height	background
text-align	dimensiones

Esta clasificación sirve como orientación. El comportamiento de herencia pertenece a la definición de cada propiedad CSS.

> Cascada y herencia son mecanismos relacionados, pero no son lo mismo.

Redefinir un estilo de forma controlada

Redefinir un estilo significa modificar una regla previa para un contexto determinado.

Clase base

```
.boton {
  padding: 0.75rem 1rem;
  border-radius: 0.5rem;
  font-weight: 600;

  background-color: #e2e8f0;
  color: #172554;
  border: 1px solid
transparent;
}
```

Variante principal

```
.boton--principal {
  background-color: #2266cc;
  color: white;
}
```

Variante secundaria

```
.boton--secundario {
  background-color:
transparent;
  color: #2266cc;
  border-color: #2266cc;
}
```

HTML resultante

```
<a class="boton boton--principal"
   href="matricula.html">
  Matricúlate
</a>
<a class="boton boton--secundario"
   href="cursos.html">
  Volver a cursos
</a>
```

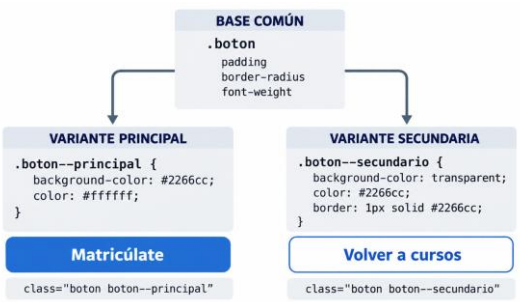
La clase `.boton` establece el estilo común y también unos valores visuales iniciales. Las clases de variante reutilizan esa base y redefinen únicamente las propiedades que deben cambiar.

Un elemento con `class="boton boton--principal"` recibe las reglas de ambas clases. Cuando ambas establecen una misma propiedad, como `background-color` o `color`, la cascada determina qué declaración prevalece.

Esta estrategia resulta más mantenible que crear selectores excesivamente específicos:

```
main section article a.boton.principal {
  ...
}
```

La redefinición debe ser intencionada y predecible, no el resultado de acumular reglas hasta conseguir que una de ellas gane.



!important: una excepción, no una estrategia

Una declaración puede marcarse como importante:

```
.boton { color: white !important; }
```

Esto modifica su prioridad dentro de la cascada. Puede resultar útil en situaciones concretas, pero utilizarlo para resolver continuamente conflictos crea hojas difíciles de mantener. Consideremos:

```
.boton {
  color: white !important;
}
.cabecera .boton {
  color: #172554;
}
```

Aunque el segundo selector es más específico dentro de las reglas normales, no puede sobrescribir esa declaración importante mediante una declaración normal. El problema suele terminar creciendo:

```
.cabecera .boton {
  color: #172554 !important;
}
```

Ahora aparecen varias declaraciones importantes compitiendo entre sí.

Antes de utilizar !important, conviene revisar:

- si el selector está apuntando al elemento correcto;
- si existe una regla demasiado específica;
- si una variante de componente resolvería mejor el problema;
- si la hoja se está cargando en el orden esperado.

No debe entenderse !important como una propiedad para «hacer que CSS funcione». Es una herramienta de la cascada cuyo uso indiscriminado dificulta la evolución del sistema de estilos.



Hojas de estilo alternativas

Un mismo documento puede disponer de más de una hoja de estilo destinada a presentar alternativas. Una forma tradicional de declararlas utiliza `title`:

```
<link rel="stylesheet"
      href="css/base.css">
<link rel="stylesheet"
      href="css/claro.css"
      title="Claro">
<link rel="alternate stylesheet"
      href="css/oscuro.css"
      title="Oscuro">
```

El navegador puede reconocer una hoja preferente y otras alternativas. Sin embargo, **no todos los navegadores actuales proporcionan al usuario una interfaz visible para cambiar entre hojas alternativas**. Por eso, en aplicaciones modernas es frecuente gestionar estas preferencias mediante otros mecanismos, por ejemplo activando una hoja concreta desde la propia interfaz o adaptando determinados estilos a las preferencias del sistema.

El concepto importante es que la estructura HTML puede mantenerse mientras cambia el conjunto de reglas de presentación aplicado.

base.css

Contiene la estructura común de componentes.

claro.css

Los valores correspondientes al tema claro.

oscuro.css

Las variantes del tema oscuro.

No es necesario duplicar todas las reglas en cada archivo. Las hojas alternativas deberían redefinir únicamente aquello que realmente cambia.



Diagnosticar CSS con las herramientas del navegador

Cuando una regla no produce el resultado esperado, modificar CSS al azar es una estrategia lenta. Las herramientas de desarrollo permiten comprobar qué está ocurriendo realmente. Al inspeccionar un elemento podemos observar:

Reglas coincidentes

Muestran qué selectores afectan al elemento.

Declaraciones sobrescritas

Permiten reconocer propiedades que han perdido frente a otra declaración.

Origen de la regla

Indica el archivo y la línea donde se encuentra.

Estilos calculados

Muestran el valor final que el navegador ha obtenido para cada propiedad.

Ante un problema de estilo conviene seguir un proceso:

01

Seleccionar el elemento correcto

Comprobar que estamos inspeccionando exactamente el nodo esperado.

02

Verificar que la regla aparece

Si no aparece, el selector no coincide o la hoja no se ha cargado.

03

Buscar declaraciones sobrescritas

Si una propiedad aparece tachada, otra declaración ha prevalecido.

04

Comparar especificidad y orden

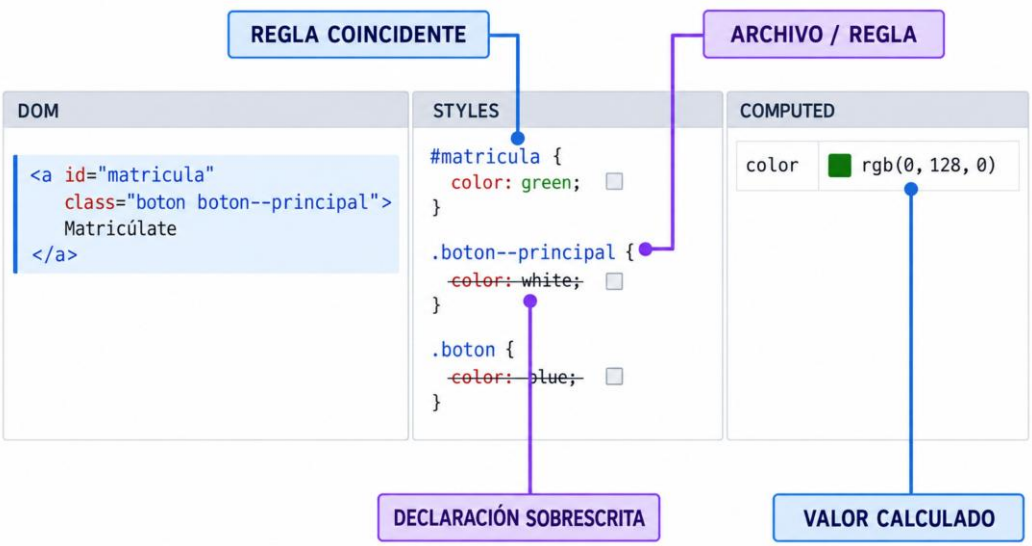
Identificar por qué ha ganado la otra regla.

05

Corregir la causa

Modificar el selector, la organización de las reglas o la variante correspondiente.

El objetivo no es conseguir que el navegador acepte nuestra última modificación. El objetivo es comprender por qué ha obtenido ese resultado.



Síntesis: controlar dónde y por qué se aplica un estilo

En este apartado hemos partido de conocimientos básicos de CSS para centrarnos en los mecanismos que permiten trabajar con hojas de estilo de forma predecible.

Formas de incorporar CSS

Los estilos pueden incorporarse directamente al elemento, dentro del documento o mediante hojas externas. En proyectos con varias páginas, las hojas externas permiten reutilizar reglas y mantener una presentación común.

Selectores

Los selectores determinan qué elementos se ven afectados. Podemos seleccionar por tipo, clase, identificador, atributo, estado o relación estructural. Las clases permiten construir reglas reutilizables y combinar un estilo común con diferentes variantes.

Cascada y especificidad

Cuando varias declaraciones intentan modificar la misma propiedad, la cascada determina cuál prevalece. La especificidad forma parte de esa decisión, pero no debe confundirse con la calidad del selector. Una especificidad innecesariamente alta dificulta las redefiniciones posteriores. El orden de aparición solo resuelve determinados empates.

Herencia y !important

La herencia permite que algunas propiedades obtengan su valor de elementos ascendentes. `!important` modifica la prioridad dentro de la cascada, pero no debe utilizarse como solución habitual a una estructura de estilos difícil de controlar.

Herramientas del navegador

Las herramientas del navegador permiten observar directamente las reglas aplicadas, los conflictos y los valores calculados.

En el siguiente apartado estudiaremos los valores, unidades, funciones y el modelo de cajas que determinan las dimensiones y el espacio ocupado por los elementos.

> Una hoja de estilos mantenible no se construye acumulando reglas hasta obtener el aspecto deseado. Se construye sabiendo qué seleccionamos, qué reutilizamos y por qué una declaración termina prevaleciendo.